UNIVERSIDADE FEDERAL RURAL DO SEMI-ÁRIDO - UFERSA
Departamento de Computação – DC
Bacharelado em Ciência da Computação
Disciplina: Segurança Computacional
Prof.: Paulo Henrique Lopes Silva

Prática Offline 1

1. Nomeação em Sistemas Distribuídos: Conceitos Fundamentais e Aplicações.

- Em qualquer sistema distribuído, há recursos (processos, arquivos, serviços, dispositivos, nós, objetos remotos) que precisam ser identificados e localizados.
- O mecanismo que permite isso é o **sistema de nomeação** ***(naming system)***.
- **Nomeação** é o processo de atribuir nomes simbólicos a recursos distribuídos e permitir que esses nomes sejam traduzidos em endereços ou identificadores reais.
- Em outras palavras:
    - **Nome**: representação simbólica compreensível (ex.: "servidor1", "db.local").
        - Um rótulo simbólico usado para referenciar algo.
    - **Endereço**: representação física (ex.: "192.168.0.10", "10.0.0.15:8080");
        - Localização física ou lógica do recurso.
    - **Identificador**: código único interno (ex.: UUID, hash, *handle*).
        - Um valor único que identifica um recurso, independente da localização.
- Esses três conceitos estão interligados, mas não são a mesma coisa.
    - Um nome pode mudar de endereço ao longo do tempo, e um mesmo endereço pode conter diferentes nomes (ex.: servidor virtualizado).

- ***Binding*** **(ligação).**

    - O *binding* é a associação entre nome, identificador e endereço.
    - Há três tipos principais:
        - **Estaticamente ligado:** Feito uma vez e nunca muda.
            - Ex.: "servidor1": 192.168.0.10" fixo no código.
        - **Ligação tardia *(late binding):*** Feita em tempo de execução, mas raramente alterada.
            - Ex.: Cliente consulta DNS a cada conexão.
        - **Ligação dinâmica *(dynamic binding)*:** Atualizada automaticamente conforme o recurso muda.
            - Serviço em nuvem que muda de IP e se registra de novo.
        - Em sistemas modernos (microsserviços, nuvem, IoT), o ***binding*** **dinâmico** é o mais usado, pois os serviços são elásticos e efêmeros.

- **Objetivos do sistema de nomeação.**

    - **Abstração da localização:**
        - O cliente não precisa saber onde o recurso está, apenas quem ele é.
    - **Transparência de mobilidade:**
        - Se o recurso se mover (mudar de IP ou servidor), o nome continua o mesmo.
    - **Escalabilidade:**
        - O sistema deve suportar bilhões de nomes e resolvê-los eficientemente (ex.: DNS global).
    - **Tolerância a falhas:**
        - O sistema deve funcionar mesmo que partes da infraestrutura falhem.
    - **Desempenho:**
        - Resolução rápida, caches e replicação de nomes para evitar gargalos.

- **Tipos de sistemas de nomeação.**

    - **Nomeação plana *(Flat naming)*.**

        - Cada recurso tem um identificador único (ex.: hash, UUID).
        - Sem estrutura hierárquica, busca é feita por difusão ou DHT.
        - Muito comum em sistemas P2P (Chord, Pastry, Kademlia).
        - Exemplo: Cada nó na rede tem um identificador SHA-1 e armazena pares chave → valor.

    - **Nomeação hierárquica.**
        - Nomes organizados em níveis (semelhante a diretórios).
        - Permite delegação de autoridade.
        - Base do DNS *(Domain Name System)*.
        - Exemplo:
        - www.instituto.ufersa.br
            - br → nível raiz.
            - ufersa → subdomínio.
            - instituto → unidade.
            - www → servidor web (Tradicionalmente aponta para o serviço HTTP/HTTPS).

    - **Nomeação baseada em contexto (ou contexto de nomes).**

        - Um mesmo nome pode significar coisas diferentes em contextos diferentes.
        - Muito usada em sistemas de objetos distribuídos (ex.: *RMI Registry*, *CORBA Naming Service*).

    - **Nomeação de conteúdo *(content-based)*.**

        - O nome é derivado do conteúdo (hash de um arquivo ou bloco de dados).
        - Usada em sistemas imutáveis e redes de conteúdo (CDNs, IPFS).
        - Exemplo:
        - O arquivo é identificado por SHA-256("conteúdo").

Questões práticas.

- **Os componentes da arquitetura devem ser executados como processos independentes.**
- **Todas as mensagens devem usar criptografia híbrida (simétrica + assimétrica).**
- **Tratamento de erros e exceções onde for possível (interpretativo).**

1. Mini-DNS para resolução de nomes (sockets UDP): Implementar um servidor simples que mantém um mapa.

```
Java
"servidor1" → "192.168.0.10"
"servidor2" → "192.168.0.20"
"servidor3" → "192.168.0.30"
"servidor4" → "192.168.0.40"
"servidor5" → "192.168.0.50"
"servidor6" → "192.168.0.60"
"servidor7" → "192.168.0.70"
"servidor8" → "192.168.0.80"
"servidor9" → "192.168.0.90"
"servidor10" → "192.168.0.100"
```

- **Casos de teste**:

    - **Caso 1**: Consulta resolução de nome (cache/mapa local).

        - Cliente 1 e Servidor DNS.

        - Fluxo:
            - Cliente 1 e DNS realizam handshake inicial para estabelecer chave simétrica (AES) via RSA e chave de integridade HMAC.
            - Cliente 1 envia requisição cifrada consultando servidor1.
            - DNS decripta, valida HMAC, busca no mapa e responde cifrado com 192.168.0.10.
        - Resultado: Cliente 1 recebe, decripta e valida o IP correto. Logs de ambos os lados registram envio, recebimento e validação de HMAC.

    - **Caso 2**: Binding dinâmico e notificação *broadcast/multicast*.

        - Cliente registrador, Servidor DNS e clientes requisitantes (C1, C2, C3).
        - Fluxo:
            - Registrador envia payload cifrado atualizando servidor1 para 192.168.0.111.
            - DNS valida o HMAC do Registrador, atualiza seu mapa e envia a notificação de atualização cifrada para C1, C2 e C3.
            - Clientes recebem a mensagem cifrada, validam HMAC e atualizam a tabela local.
        - Resultado: Todos os clientes imprimem a atualização de IP recebida do DNS com sucesso após validação criptográfica.

    - **Caso 3**: Tratamento de entrada inválida e erro de autenticação.

        - Cliente 2 e Servidor DNS.
        - Fluxo: Cliente 2 envia nome inexistente (servidor99) ou requisição com HMAC corrompido.
        - Resultado:
            - Nome inexistente / Formato inválido: DNS rejeita e responde/imprime Erro na entrada de dados. Tente outra vez!.
            - Falha de HMAC/Cifra: DNS descarta o pacote UDP sem processar e registra falha de integridade no log.

2. Descoberta de serviços *(service discovery)*: Criar servidores que oferecem um serviço simples (ex.: “Calcular soma”, “Calcular subtração”, “Calcular multiplicação” e “Calcular divisão”).

- Três servidores possuem o serviço de calculadora básica, com as operações listadas (cada qual com seu nome e endereço simulados).
- Um servidor de diretório registra todos os servidores disponíveis.
- Considere três clientes:
    - Um cliente consulta o diretório para descobrir qual servidor está ativo.
    - Faça testes de descoberta com, pelo menos, dois serviços.
- **Casos de teste**:

    - **Caso 1**: Descoberta de serviço e cifra híbrida no diretório.

        - Cliente 1, Servidor de diretório e Servidores de Calc (S1, S2, S3).
        - Fluxo:
            - S1, S2 e S3 registram seus serviços no Diretório usando pacotes cifrados (RSA+AES+HMAC).
            - Cliente 1 estabelece canal seguro com o Diretório e solicita o serviço "Calcular soma".
            - Diretório responde com a lista de servidores ativos cifrada.
        - Resultado: Cliente 1 obtém a lista de endereços autenticada e grava logs do canal seguro.

    - **Caso 2**: Chamada de serviço com *load balancing* (round robin / random).

        - Cliente 1, Cliente 2 e Servidores de Calc (S1, S2).
        - Fluxo:
            - Cliente 1 e Cliente 2 fazem requisições sequenciais de cálculo (10 + 20, 5 * 4).
            - A política de balanceamento (ex.: round robin) alterna a conexão para S1 na 1ª chamada e S2 na 2^a.
            - Cada chamada abre/usa sessão cifrada (AES+HMAC) individual com o servidor selecionado.
        - Resultado: O cálculo é processado no nó correto com canal seguro; logs demonstram a distribuição proporcional de conexões.

    - **Caso 3**: Servidor inativo ou falha de integridade.

        - Cliente 3, Diretório e Servidor de Calc (S3 indisponível/adulterado).
        - Fluxo:
            - Cliente tenta se conectar a um serviço retornado cujo HMAC falha na negociação ou cuja entrada na consulta é inválida (ex.: operação "Calcular modulo" não cadastrada).
        - Resultado:
            - Entrada inválida: Imprime Erro na entrada de dados. Tente outra vez!.
            - HMAC corrompido/Falha de conexão: Rejeita a execução e aciona o tratamento de erro sem travar os demais componentes.
-

3. Nomeação em P2P (*overlay* simplificado): Desenvolva uma aplicação cliente/servidor *multithread*, com TCP ou UDP, para simular a comunicação de processos (nós) em uma topologia em anel. Considere seis processos chamados de P0, P1, P2, P3, P4 e P5. Considere o seguinte esquema:

- Cada nó tem um identificador numérico.
- O nó sabe quem é seu sucessor e seu antecessor.
- Para encontrar um recurso (ex.: arquivo “X”), um nó repassa a mensagem ao próximo até o responsável ser encontrado.
    - Os arquivos podem ser simulados com strings: arquivo1.ext, arquivo2.ext, arquivo23.ext, arquivo47.ext, etc.
    - A rede armazena 60 arquivos, sendo:
        - P0 com os arquivos de 1 a 10.
        - P1 com os arquivos de 11 a 20.
        - P2 com os arquivos de 21 a 30.
        - P3 com os arquivos de 31 a 40.
        - P4 com os arquivos de 41 a 50.
        - P5 com os arquivos de 51 a 60.
- Esse exercício mostra nomeação distribuída sem servidor central.
- **Casos de teste**:

    - **Caso 1**: Busca válida com criptografia salto a salto.

        - Origem: P0 | Arquivo: arquivo35.ext
        - Fluxo (Horário): P0 → P1 → P2 → P3.
        - Resultado: Cada enlace (P0–P1, P1–P2, P2–P3) estabelece canal seguro com troca de chaves AES via RSA.
        - P0 encripta com AES/HMAC e envia a P1;
        - P1 e P2 decriptam, verificam integridade, não encontram localmente e re-encriptam para o próximo nó.
        - P3 decripta, localiza o arquivo (faixa 31–40) e responde com mensagem cifrada a P0.
        - Nós registram em log o tráfego e a validação criptográfica.

- **Caso 2**: Entrada inválida.
    - Origem: P1 (arquivo99.ext) e P4 (arquivoABC.ext).
    - Resultado:O nó valida o comando antes de iniciar criptografia ou abrir conexões TCP/UDP.
    - Rejeita a busca sem propagar pacotes pelo anel.
    - Imprime no console: Erro na entrada de dados. Tente outra vez!.

- **Caso 3**: Falha de integridade/Pacote adulterado.
    - Origem: P4 enviando para P5.
    - Cenário: Mensagem cifrada alterada no canal ou HMAC inválido.
    - Resultado: P5 detecta a divergência do HMAC, descarta o pacote e interrompe o repasse para P0.
    - P5 grava no log a falha de autenticação/integridade.

4. CDN: Uma *Content Delivery Network* (CDN) é uma infraestrutura distribuída composta por servidores estrategicamente posicionados (*Edge Servers* ou *Points of Presence - PoPs*) interligados pela internet. O objetivo fundamental é aproximar o conteúdo do usuário final, reduzindo a latência, otimizando o consumo de banda e aumentando a resiliência do serviço.

- Aplicações reais de CDNs:
    - *Streaming*: Entrega mídia em blocos (.ts/.m4s) para reduzir latência e evitar buffering (ex.: Netflix, Spotify).
    - Downloads grandes: Suporta picos de tráfego em jogos e atualizações de SO (ex.: Steam, PSN).
    - Aceleração Web: Salva recursos estáticos (CSS, JS, imagens) na borda para carregamento rápido (ex.: e-commerce).
    - Proteção DDoS: Absorve ataques volumétricos na borda, protegendo o servidor de origem.

4.1. Simulação: Arquitetura da CDN baseada em Sockets.

- Para simular o funcionamento interno de uma CDN com sockets, o sistema deve ser dividido em quatro componentes principais:

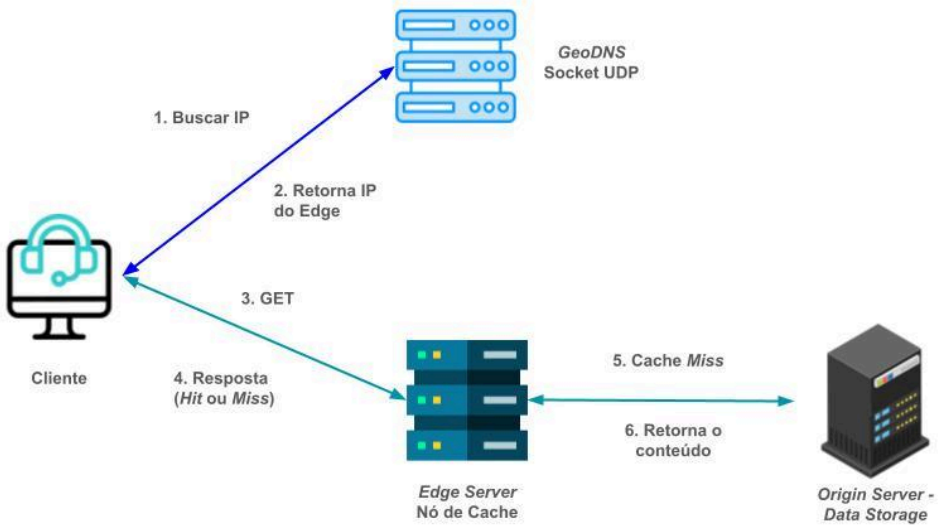


Figura 1: Arquitetura simplificada de uma CDN.

- *Origin Server* (Servidor de Origem).
    - Socket: TCP escutando em uma porta fixa (ex: 8000).
    - Papel: Armazena o banco de dados/arquivos mestre.
    - Lógica: Recebe uma requisição socket, busca o arquivo local no disco e responde o conteúdo bruto.
    - O servidor armazena 1000 arquivos (pode usar a mesma nomenclatura da questão 3).
- *Edge Servers / Cache Nodes* (Nós de Borda).
    - Socket:
        - Servidor TCP: Escuta requisições dos clientes (ex: portas 8001, 8002, 8003).
        - Cliente TCP: Abre uma conexão com o servidor de origem em caso de *cache miss*.
    - Lógica:
        - Mantém uma tabela hash/dicionário em memória (cache).
        - Ao receber a mensagem do cliente:
            - *Cache hit:* Retorna o dado imediatamente.
            - *Cache miss:* Abre socket com a Origem, baixa o dado, salva na memória/disco local e envia de volta ao cliente.
- *Router / GeoDNS* (Direcionador de Tráfego).
    - Socket: UDP ou TCP escutando numa porta principal (ex: 5353).
    - Lógica:
        - Recebe a solicitação do Cliente: "Onde baixo o arquivo X?".
        - Avalia a localização fictícia do Cliente (enviada no *payload*).
        - Responde ao cliente com o IP/Porta do *Edge Server*.
- Cliente (Gerador de Tráfego).
    - Socket: Cliente TCP/UDP.
    - Lógica:
        - Consulta o *Router* para descobrir a qual *Edge* se conectar.
        - Estabelece conexão socket TCP com o *Edge* fornecido.

- **Protocolo de aplicação simplificado.**
    - Em vez de usar HTTP completo, você pode definir um protocolo simples baseado em texto/JSON transmitido sobre os sockets TCP:
        - Requisição do Cliente para o *Edge*:
            - GET /arquivo12.ext
            - Resposta do *Edge* (cache *hit*):
                - 200 OK
                - Cache: HIT
                - Length: 1024
            - Notificação de Invalidação *(Origin -> Edges)*:
                - INVALIDATE /arquivo12.ext

- **Casos de teste.**
    - Para validar o funcionamento da CDN, aplique cenários de teste que demonstrem algumas propriedades de sistemas distribuídos:
        - **Caso 1: Validação de cache HIT vs. cache MISS em canal cifrado.**

- Cliente, *Router/GeoDNS*, *Edge* e Servidor de Origem.
- Fluxo:
    - Cliente consulta *Router* (via UDP cifrado) e recebe IP/Porta do *Edge*.
    - Primeira requisição (GET /arquivo20.ext): *Edge* verifica Cache MISS. Abre socket TCP cifrado com a Origem, baixa o arquivo via AES+HMAC, salva em cache e envia cifrado ao Cliente.
    - Segunda requisição (GET /arquivo20.ext): *Edge* identifica Cache HIT. Responde imediatamente ao cliente com o dado do cache local cifrado.
    - Resultado:
        - A segunda resposta apresenta tempo menor na prática;
        - logs registram HIT e MISS com HMAC validado nas duas pontas.

- **Caso 2: Expiração por TTL e invalidação ativa (INVALIDATE).**

    - Servidor de Origem, *Edge* e Cliente.
    - Fluxo: Arquivo em cache expira após TTL (ex.: 10s) ou a Origem envia comando INVALIDATE /arquivo30.ext cifrado via TCP para os *Edges*.
    - *Edge* recebe, decripta a invalidação com a chave da Origem, checa HMAC e remove arquivo30.ext do cache.
    - Próxima requisição do Cliente força um Cache MISS, obrigando nova busca segura na Origem.
    - Resultado: Consistência estrita garantida e logs confirmam a invalidação e renovação segura do cache.

- **Caso 3: Requisição inexistente / Erro na cifra da borda.**

    - Cliente e *Edge*.
    - Fluxo: Cliente envia pedido de arquivo com nome fora do padrão/inválido (GET /arquivoXYZ.ext) ou ocorre erro de decifragem no handshake.
    - Resultado:
        - Formato/arquivo inválido: Sistema não consulta a Origem e retorna/imprime Erro na entrada de dados. Tente outra vez!.
        - Envio de mensagem sem o HMAC correto (ou usando um hash sem chave): O *Edge* encerra a conexão de socket com o cliente, emitindo alerta no log sem corromper a tabela de cache.

Observações

- Data e hora de entrega da tarefa: 20/10/2026 até às 23h59. Portanto, certifiquem-se do arquivo que vão enviar.
- Crie uma pasta chamada “seu-nome-pratica-off1”, comprima e envie pelo SIGAA (única forma de envio).
- Avaliação: o projeto vale 100% da nota da 2ª unidade.
- Os projetos podem ser feitos de forma individual ou em dupla.
- Os projetos devem utilizar as tecnologias vistas na disciplina (*Threads*, *Sockets* TCP e UDP).
- Casos de punição com a nota zero, para os envolvidos.
    - Respostas ou códigos iguais.
    - Respostas ou códigos sem a devida explicação.
- Bom trabalho!

Roteiro para Gravação da Apresentação

1. Executar o programa resultante de cada enunciado, questão ou projeto.
2. Para cada programa, execute uma operação por vez.
    - Durante a testagem de cada funcionalidade, comente sobre os possíveis erros e exceções que podem ocorrer mediante entradas incorretas. Comente também sobre quais erros e exceções seu programa captura.
    - Cite as ferramentas e tecnologias utilizadas.
    - Seu programa implementa todas as funcionalidades requeridas? Em caso negativo, justifique.
3. Após a demonstração de funcionamento, é hora de abrir o código.
    - Explique a organização das respostas.
    - Houve alguma dificuldade ou detalhe interessante na implementação de algum dos requisitos mencionados?
    - Quais as principais dificuldades e lições aprendidas com o desenvolvimento do projeto?
    - Responda as perguntas detalhando o código que você implementou.
4. Detalhamento das instruções:
    - O vídeo deve ter no máximo 8 minutos.
    - A gravação será o instrumento avaliativo.
    - Pontos de avaliação:
        - Sequência lógica e domínio do conteúdo (introdução, divisão organizada da apresentação, concatenação de ideias e conclusão).
        - Precisão na comunicação (colocação e entonação da voz, ritmo, dicção e linguagem).
        - Capacidade de argumentação.
        - Domínio e uso de material (organização no uso de recursos de apresentação).
5. Sugestões de software para gravação:
    - Google Meet, Zoom, SimpleScreenRecorder, OBS, etc.